Ferramenta de Conformidade de Hardening para Red Hat Enterprise Linux

Documento de escopo do projeto – Disciplina de Sistemas Operacionais

Projeto	Ferramenta de linha de comando de conformidade de configuração (hardening) – nome a definir
Disciplina	Sistemas Operacionais – [Instituição / Curso / Período]
Equipe	[Aluno A], [Aluno B], [Aluno C]
Parte interessada externa	ATI – Agência Estadual de Tecnologia da Informação (Recife/PE), por meio de gerente que validou a ideia
Prazo	Entrega em 09/11/2026 (16 dias a partir de 24/10/2026)
Versão / data	1.0 – [data]

1. Contexto e problema

Hardening de sistemas é um tema amplo: abrange configuração de contas, rede, kernel, sistema de arquivos, serviços, auditoria e muito mais. Organizações costumam conhecer referências como o CIS Benchmarks e o NIST SP 800-123, mas aplicá-las de forma consistente e repetível em muitos servidores é difícil. Segundo a ATI, essas referências são conhecidas, mas sua aplicação não é normatizada.

O projeto propõe uma ferramenta que transforma uma baseline de hardening em uma **política declarativa**, verifica a conformidade do servidor, aplica as correções de forma segura e gera evidências. A ferramenta **não é um antivírus nem um scanner de vulnerabilidades**: ela aplica regras estritas de configuração.

2. Retorno da ATI

A ideia foi validada pelo gerente da ATI, que considerou útil ter o resultado por meio de ferramenta ou script. As informações relevantes para o escopo foram:

- **Ambiente:** há ambientes Red Hat OpenShift, mas não há conhecimento de ambientes com RHEL puro. Mais de 95% do parque Linux é Ubuntu, com alguns CentOS.
- **Referências:** a ATI conhece o CIS Benchmarks e o NIST SP 800-123, sem aplicação normatizada.

Dores priorizadas pela ATI:

#	Dor relatada
1	Quais configurações de SO mitigam ou bloqueiam ataques de ransomware.
2	Como evitar que áreas de upload de arquivos por usuários web sejam usadas de forma maliciosa (já houve ameaças para Windows armazenadas em máquina Linux e disponibilizadas para download).
3	Bloqueios de comunicação para evitar movimentação lateral de ameaças.
4	Ao aplicar patches, é comum a aplicação parar de funcionar e precisar de correções.
5	Controle de credenciais: revogação em curto prazo para pessoas desligadas.
6	Mínimo privilégio de acesso.

3. Objetivo e princípios

Objetivo geral: desenvolver, até 09/11/2026, uma ferramenta de conformidade de configuração para Linux que verifique, aplique e reverta regras de hardening definidas em política, priorizando as dores relatadas pela ATI.

Princípios de projeto:

- **Conformidade, não detecção:** o foco é estado de configuração, não malware nem CVEs.
- **Política declarativa:** regras em YAML, legíveis por humanos e rastreáveis ao CIS Benchmark.
- **Segurança operacional:** modo simulação (dry-run), backup antes de alterar, validação de sintaxe antes de recarregar serviços e rollback.
- **Idempotência:** executar duas vezes produz o mesmo resultado.
- **Leveza:** sem servidor central; um único programa Python com dependência mínima (PyYAML).
- **Evidência:** relatório por regra e por dor da ATI, em texto e JSON.

4. Plataforma-alvo

A disciplina exige **Red Hat Linux**, portanto a ferramenta será desenvolvida e testada exclusivamente na família RHEL. A ATI informou que seu parque Linux é majoritariamente Ubuntu, com alguns CentOS, e que não há conhecimento de RHEL puro; por isso o documento registra de forma transparente o alcance do projeto.

Item	Definição
Alvo principal	RHEL 9 (ou Rocky/AlmaLinux 9, equivalentes binários gratuitos, caso não haja assinatura). Gerenciador **dnf**, firewall **firewalld**, MAC **SELinux**, Python 3.9 do sistema.
Relação com a ATI	Os CentOS existentes no parque pertencem à mesma família (dnf/yum, firewalld, SELinux); as regras são, portanto, aplicáveis a eles com poucos ajustes. Para o RHEL, o projeto serve como baseline de referência caso a ATI venha a ter ambientes RHEL.
Ubuntu	**Fora do escopo.** As regras serão escritas de forma modular e referenciadas ao CIS, de modo que uma futura adaptação para Ubuntu (apt, ufw/nftables, AppArmor) seja possível, mas não será implementada nem testada.

Observação: o CentOS 7 já está sem suporte e traz Python 3.6; compatibilidade com ele não é objetivo, mas evitar recursos exclusivos de Python recente facilita a portabilidade.

5. Escopo funcional: das dores aos módulos

Cada dor da ATI foi traduzida em verificações e correções de configuração no host. A tabela também registra os limites honestos do que um hardening de host consegue fazer.

Dor	Entregável concreto em RHEL 9	Qtd.	Limite
5. Revogação de credenciais	Comando *offboarding --autorizados lista.csv*: bloqueia contas locais fora da lista (usermod -L, chage -E 0, shell nologin), arquiva authorized_keys, remove de wheel e encerra sessões. Também reporta contas inativas, senhas sem expiração e chaves sem dono.	~8	Identidades em AD/LDAP são revogadas no diretório, não no host.
6. Mínimo privilégio	sudoers e /etc/sudoers.d sem NOPASSWD/ALL indevidos; wheel conferido com a lista autorizada; UID 0 único; PermitRootLogin no; SUID/SGID comparados com lista permitida; permissões de /etc/shadow, /etc/passwd e sshd_config; umask; faillock e pwquality.	~8	—

Dor	Entregável concreto em RHEL 9	Qtd.	Limite
3. Movimentação lateral	firewalld com zona restritiva e só os serviços declarados; portas em escuta (ss) comparadas com a lista autorizada; AllowGroups no SSH; sysctl de rede (ip_forward, redirects, rp_filter, log_martians).	~6	Segmentação de rede (VLANs, firewall de perímetro) não é do host.
1. Ransomware	noexec,nosuid,nodev em /tmp, /var/tmp e /dev/shm; SELinux em enforcing; arquivos world-writable sem sticky bit e sem dono; serviços inseguros desativados (telnet, rsh, tftp etc.); sysctl de kernel (randomize_va_space, kptr_restrict, dmesg_restrict, fs.suid_dumpable).	~5	Nenhuma configuração bloqueia ransomware sozinha: reduz superfície de ataque e raio de impacto.
2. Uploads	Diretórios de upload declarados na política: sem bit de execução, dono e modo corretos, montagem separada noexec,nosuid,nodev (ou ao menos fora do webroot), contexto SELinux sem permissão de execução e verificação de que o Apache não executa scripts no caminho (-ExecCGI, PHP desligado).	~4	Não inspeciona o conteúdo dos arquivos: evita que o servidor seja comprometido por esse caminho.
4. Patches (fora da entrega)	Não será implementada no prazo. O design fica documentado como trabalho futuro: snapshot do estado antes da atualização (pacotes, serviços ativos, portas, hash de /etc), health checks depois e rollback das configurações gerenciadas, com apoio do dnf history.	—	Por causa do prazo de 16 dias; versão mínima de snapshot/diff só se sobrar tempo.

Total estimado: cerca de 30 regras, viáveis para três pessoas em 16 dias se o contrato entre módulos for fechado no primeiro fim de semana.

6. Premissas do projeto

A resposta da ATI já traz as dores, a referência (CIS) e a confirmação de que uma ferramenta ou script resolve. Para não depender de novas consultas, os pontos em aberto foram decididos pela equipe e passam a ser premissas do projeto:

Ponto em aberto	Decisão do projeto	Por que não depende da ATI
Contas locais ou LDAP/AD?	**Contas locais.**	Existem em qualquer servidor. A revogação em diretório fica fora do escopo e declarada como limite.
Qual servidor web nos uploads?	Regras no **nível de SO** (permissões, montagem, SELinux), independentes do servidor web; como exemplo concreto, **Apache httpd**, o padrão do RHEL.	As proteções mais fortes não dependem da aplicação.
Origem da lista de desligados?	Entrada em **CSV simples** (coluna *usuario*), definido pela equipe.	Qualquer fonte (RH, AD, planilha) pode ser exportada para esse formato.
Ambiente para testar?	VM RHEL 9 (ou Rocky/Alma 9) com snapshot e **script de demonstração** que cria as violações.	Não há ambiente da ATI disponível; a prova é reprodutível por qualquer pessoa.

7. Priorização

Com 16 dias até a entrega, o escopo foi reduzido para caber com folga e permitir testes reais. A prioridade é entregar o essencial bem testado no RHEL.

Nível	Conteúdo
MVP (obrigatório)	Dores 5, 6 e 3, mais as partes de SO das dores 1 e 2 (cerca de 30 regras); modos audit, enforce --dry-run, enforce e rollback; relatório em texto; offboarding por CSV; script de demonstração; RHEL 9.
Desejável	Relatório em JSON agrupado por dor; verificação aprofundada do SELinux; regras do Apache httpd para uploads; comparação antes/depois com OpenSCAP.
Trabalho futuro	Guarda de patch (dor 4) com design documentado; controle de saída (egress) no firewalld; auditd; adaptação para CentOS e Ubuntu.

8. Arquitetura proposta

- **Linguagem:** Python 3.9+; política em YAML (PyYAML).
- **Modos da CLI:** *audit* (apenas verifica), *enforce --dry-run* (mostra o que mudaria), *enforce* (aplica), *rollback* (desfaz) e *report* (gera relatório).
- **Estrutura do repositório:** core/ (motor), rules/ (YAML por domínio), lib/ (primitivas), tests/ e docs/. Chamadas específicas do RHEL ficam isoladas em uma camada única, facilitando uma futura portabilidade.
- **Primitivas reutilizáveis:** chave do sshd_config, sysctl, permissões de arquivo, opções de montagem, pacote, serviço, usuário/grupo, sudoers, portas em escuta e firewall.
- **Segurança operacional:** backup antes de cada mudança; validação (*sshd -t*, *visudo -c*) antes de recarregar; manter sessão de acesso aberta ao aplicar regras de SSH ou firewall.

Formato de uma regra (exemplo):

```
- id: SSH-001
  descricao: Desabilitar login direto como root
  referencia: CIS (item a mapear) | dor_ati: [6]
  severidade: alta
  plataforma: rhel9
  verificar: sshd -T | grep -i permitrootlogin
  esperado: permitrootlogin no
  corrigir: arquivo próprio em /etc/ssh/sshd_config.d/
  recarregar: sshd
```

Ambiente de desenvolvimento

- **VM de teste:** uma VM RHEL 9 (ou Rocky/AlmaLinux 9) hospedada na máquina de um integrante, com snapshot limpo (*base-limpa*) e acesso dos demais por SSH.
- **Desenvolvimento local:** quem não hospeda a VM escreve código, YAML e testes unitários (pytest, com mocks e diretórios temporários) na própria máquina e valida na VM por *git pull*.
- **Regra de segurança:** o comando *enforce* só é executado na VM, nunca nas máquinas pessoais.
- **Fila de validação:** o que depende do sistema real (SELinux, firewalld, sysctl, montagens, contas) é listado em um arquivo *VALIDAR_NA_VM.md* e executado em lote pelo responsável pela VM.
- **Containers:** um container Rocky/Alma 9 pode servir para testes unitários, mas não reproduz kernel, SELinux nem firewalld do host e não substitui a VM.

Uso de IA no desenvolvimento

- **Contrato primeiro:** esquema YAML da regra, interface das primitivas e formato do relatório definidos pelos três antes de gerar código.
- **Contexto no repositório:** arquivo de contexto com objetivo, escopo, contrato e convenções (Python 3.9, só PyYAML, dry-run por padrão).
- **Onde a IA é usada:** esqueleto da CLI, primitivas repetitivas, rascunho de regras, testes, script de demonstração e documentação.
- **Ciclo por regra:** audit reprova na VM suja, enforce corrige, segundo enforce não altera nada, rollback restaura, commit.
- **Cuidados:** revisar todo diff antes de executar como root; confirmar cada item do CIS no benchmark oficial, sem confiar em numeração de memória; registrar as partes com apoio de IA conforme a política da disciplina; todos devem saber explicar o código.

9. Divisão da equipe

Aluno	Trilha	Responsabilidades
A	Núcleo	CLI, leitor de política, backup e rollback, relatório, testes; depois script de demonstração e documentação.
B	Identidade e privilégio (dores 5 e 6)	Offboarding, sudoers, SSH, contas, PAM, SUID/SGID.
C	Host, rede e dados (dores 1, 2 e 3)	Montagens, firewalld, portas em escuta, sysctl, regras de upload, SELinux.

Cada integrante escreve o código localmente e valida na VM RHEL (com snapshot), hospedada na máquina de um integrante. O Aluno A entrega cedo a primitiva mínima para não bloquear os demais.

10. Cronograma

Período	Entregas
24 a 25/10 (sáb-dom)	Contrato entre módulos (esquema YAML, interface das primitivas, formato do relatório), repositório, VM RHEL com snapshot, uma regra ponta a ponta (YAML, audit, relatório).
26 a 30/10	Audit das cerca de 30 regras e relatório. Checkpoint em 28/10: se o audit não estiver rodando, corta-se escopo.
31/10 e 01/11	Enforce com backup e rollback, começando pelas regras mais seguras.
02 a 05/11	Offboarding, regras de upload, script de demonstração e testes. **Congelamento do código em 05/11.** (02/11 é feriado.)
06 a 08/11	Comparação com OpenSCAP antes/depois, documentação, slides e ensaio da demonstração.
09/11	Entrega.

Regra de ouro: nenhuma funcionalidade nova depois de 05/11.

11. Plano para o primeiro fim de semana (24 e 25/10)

Dia	Atividades
Sábado	Reunião de 1 a 2 horas para fechar o contrato entre módulos e as três trilhas; criar o repositório com o arquivo de contexto para IA; o integrante com mais espaço instala a VM RHEL 9 (ou Rocky/Alma 9), tira o snapshot limpo e libera o acesso SSH; os demais preparam o ambiente local (Python 3.9+, PyYAML, pytest, editor); agradecer ao gerente da ATI pelo retorno.
Domingo	Aluno A: esqueleto da CLI, leitor de YAML e a interface base de verificação, com testes. Aluno B: 5 regras de SSH/sudoers e formato do CSV de offboarding. Aluno C: 5 regras de montagem/sysctl e lista de caminhos típicos de upload. Todos: tabela dor / regra / item CIS (cerca de 30 regras).
Meta	Uma regra rodando ponta a ponta (YAML, audit, relatório) na VM até domingo à noite.

12. Validação, demonstração e critérios de sucesso

- VMs “sujas” (fora da baseline) passam a conformes após o enforce, com relatório comprovando a mudança.
- Segunda execução do enforce não altera nada (idempotência).
- Rollback restaura o estado anterior das configurações alteradas.
- No RHEL, o resultado é comparado com o OpenSCAP usando um perfil CIS, antes e depois.
- As regras ficam modulares e referenciadas ao CIS, com o código específico do RHEL isolado.
- Cada regra possui referência ao CIS Benchmark e vínculo com ao menos uma dor da ATI.

Roteiro de demonstração

Como não haverá ambiente da ATI para testar, a equipe criará o script *sujar_servidor.sh*, que cria deliberadamente as violações na VM: usuário fora da lista com chave SSH, NOPASSWD no sudoers, /tmp sem noexec, diretório de upload com executáveis, porta extra em escuta e SELinux em permissive. A sequência de demonstração é:

Passo	Ação	Resultado esperado
1	*audit*	Relatório "antes": violações listadas por regra e por dor da ATI.
2	*enforce*	Correções aplicadas, com backup prévio.
3	*audit*	Relatório "depois": tudo conforme.
4	*rollback*	Estado anterior restaurado.
5	OpenSCAP (perfil CIS)	Comparação independente antes/depois.

13. O que o projeto não promete

- **Bloquear ransomware:** reduz superfície de ataque e raio de impacto; não substitui backup, EDR ou segmentação.
- **Detectar malware nos uploads:** evita que o servidor seja comprometido por esse caminho, mas não inspeciona o conteúdo.
- **Revogar identidades em AD/LDAP** e **segmentar a rede:** são camadas fora do host.
- **Proteger contra falhas de patches:** a guarda de patch (dor 4) ficou fora da entrega por causa do prazo; o design fica documentado como trabalho futuro.
- **Funcionar no Ubuntu:** fica como trabalho futuro, facilitado pelo mapeamento ao CIS. Os CentOS da ATI são da mesma família do RHEL, mas não serão testados.

14. Riscos e mitigações

Risco	Mitigação
Regras de SSH, PAM ou firewall trancarem o acesso ao servidor	Testar apenas em VM com snapshot; manter sessão aberta; validar sintaxe antes de recarregar; dry-run por padrão.
Prazo curto (16 dias)	Escopo reduzido a cerca de 30 regras; guarda de patch fora da entrega; checkpoint em 28/10; congelamento do código em 05/11.
VM em apenas uma máquina (espaço e disponibilidade)	Desenvolvimento local com testes unitários e mocks; validações em lote na VM; snapshots frequentes; plano B: instância temporária em nuvem (atenção a custos).
Parque da ATI é majoritariamente Ubuntu, não RHEL	Registrar o alcance de forma transparente; destacar aplicabilidade aos CentOS (mesma família) e a referência ao CIS para futura portabilidade.
SELinux bloquear serviços legítimos ao endurecer permissões	Manter SELinux em enforcing nos testes, usar logs do audit para ajustar e documentar exceções por regra.
Expectativa de que a ferramenta “bloqueie ransomware”	Comunicar o limite: reduz superfície e impacto; não substitui backup, EDR ou segmentação.
Sem ambiente da ATI para validar	Script de demonstração que cria as violações, relatórios antes/depois e comparação com OpenSCAP: prova reprodutível por qualquer pessoa.
Regras do CIS inadequadas a algum serviço da ATI	Política configurável com exceções documentadas por regra.

15. Valor entregue à ATI

Ao final, a ATI recebe uma **baseline de hardening em RHEL 9, auditável e reversível, mapeada às suas seis dores e ao CIS**. Mesmo que o parque seja majoritariamente Ubuntu, as decisões de política (o que bloquear, em que ordem e com quais exceções) são reaproveitáveis, a lógica de verificação por regra serve de modelo e a ferramenta funciona como protótipo para uma versão futura. O repositório incluirá o script de demonstração, para que a ATI reproduza a prova sem depender da equipe.

16. Referências

- CIS Benchmark para Red Hat Enterprise Linux 9 (Center for Internet Security).
- NIST SP 800-123: Guide to General Server Security.
- OpenSCAP / SCAP Security Guide (usados apenas para validação comparativa).